

LAB_2 - Revision of Basic Data Structures

A2 - S4 - 2025/2026 - II.2415

Advanced Algorithms and Programming

Team 04

Damiano Spadaccini - Nicolò Romanelli

GitHub repository: <https://github.com/damspada/AAP-T04-Semester-Project> (branch LAB_02_Revision, folder LAB_02)

This is the final version of our LAB_2 report. We kept the pseudo-code and the answers of the report written during the session (T04_LAB2_AES). For each exercise we added more complexity analysis, a short reflection and the tests of our Python code. We also added the pseudo-code of ReduceMatrix (bonus of exercise 3) and the final question.

The code is on GitHub (link above). Each file can be run alone and prints the result of every test.

EXERCISE 1 - FRIEND REQUEST TIMELINE

Function AnalyzeRequest(msg: String) -> (Integer, Integer, Real, String, Boolean)

```
upper <- 0
alpha <- 0
punct <- 0
run <- 0
max_run <- 0
// no previous letter yet
prev <- ""

For each c in msg Do
  If IsLetter(c) Then
    // count the letters and the uppercase letters
    alpha <- alpha + 1
    If IsUppercase(c) Then
      upper <- upper + 1
    End If
    // length of the current run of equal letters, like "yyyy" in "heyyyy"
    If ToLower(c) = prev Then
      run <- run + 1
    Else
      run <- 1
      prev <- ToLower(c)
    End If
    If run > max_run Then
      max_run <- run
    End If
  Else
    // only letters make a run: "!!!!" is already counted as punctuation
    run <- 0
    prev <- ""
    // count the urgency punctuation
    If c = "!" Or c = "?" Then
      punct <- punct + 1
    End If
  End If
End For

// avoid the division by zero when there are no letters
If alpha > 0 Then
  caps_ratio <- upper / alpha
Else
  caps_ratio <- 0
End If
```

```

// from the strongest to the weakest condition
If caps_ratio >= 0.6 Or punct >= 5 Then
  label <- "AGGRESSIVE"
Else If caps_ratio >= 0.3 Or punct >= 3 Then
  label <- "URGENT"
Else
  label <- "CALM"
End If

// more than 3 equal letters in a row
spam <- max_run > 3
Return (upper, punct, caps_ratio, label, spam)
End

```

1. How many character comparisons are needed for an n-character string?

Each character is checked at most 3 times: IsLetter, then IsUppercase and the comparison with prev if it is a letter, or the tests on "!" and "?" if it is not (IsLetter and IsUppercase count as one test each). So there are at most $3n$ comparisons, and the total grows linearly: $O(n)$.

2. Can this be done in a single pass? What is the minimal number of passes required?

Yes, one pass is enough. We update all the counters while reading the string once. One pass is also the minimum, because we must read every character at least once to know the answer.

3. How would the complexity change if we needed to support Unicode characters beyond ASCII?

The complexity stays $O(n)$, but each check is slower. In UTF-8 a character can take from 1 to 4 bytes, so the program has to decode the string first. IsLetter and IsUppercase have to look up the Unicode tables instead of doing a simple comparison, and ToLower does not always give one character for one character (for example "ß" becomes "SS" in uppercase). Also, some languages have no uppercase letters, so the caps ratio is less meaningful for them.

More complexity analysis and reflection

The algorithm reads the whole message once, so the time is $\Theta(n)$ in the best and in the worst case. The memory is $O(1)$, because we only use a few counters. In our code we also counted the character comparisons: a "!" costs 2 comparisons and every other character costs 3, so the total is always about $3n$, as in the last column of the table.

The rules of the lab overlap: for example a caps ratio of 0.7 is both URGENT and AGGRESSIVE, so we check AGGRESSIVE first. We also saw that the caps ratio does not work well on very short messages: "Sì" is AGGRESSIVE only because it has two uppercase letters.

Test set

All the tests print OK. Our numbers for the examples are a bit different from the lab text, because we count every uppercase letter and we divide by all the letters, as the formula says. The labels are the same.

Case	Message	Upper	Punct	Caps ratio	Label	Spam	Comparisons
example 1	"Hey, want to connect?"	1	1	0.06	CALM	False	63
example 2	"PLEASE ACCEPT MY REQUEST!!!"	21	3	1.00	AGGRESSIVE	False	78
example 3	"Are you free? I need to talk!!!"	2	4	0.10	URGENT	False	90
empty message	""	0	0	0	CALM	False	0
no letters	"?!?!?"	0	5	0	AGGRESSIVE	False	13
caps ratio = 0.3	"PIZZa buona"	3	0	0.30	URGENT	False	33
caps ratio = 0.6	"FORza"	3	0	0.60	AGGRESSIVE	False	15
short message	"Sì"	2	0	1.00	AGGRESSIVE	False	6
repeated letters	"ciaooooo"	0	0	0	CALM	True	24
only 3 equal letters	"grazieee"	0	0	0	CALM	False	24
mixed case run	"BellSSSsimo"	4	0	0.33	URGENT	True	36

Case	Message	Upper	Punct	Caps ratio	Label	Spam	Comparisons
space in the run	"Daiii iii"	1	0	0.13	CALM	False	27
spam and urgent	"Carbonaaaaa!!!"	1	3	0.08	URGENT	True	42
letter with accent	"PERCHÉ?"	6	1	1.00	AGGRESSIVE	False	21

EXERCISE 2 - MUTUAL FRIENDS DETECTION USING SETS

The sets are hash sets, so Contains and Add cost $O(1)$ on average, and Add ignores an element that is already in the set.

Function Intersection(A: Set, B: Set) -> Set

```
R <- empty set
// walk the smaller set and look for its elements in the bigger one
If Size(A) > Size(B) Then
    // swap
    temp <- A
    A <- B
    B <- temp
End If
For each x in A Do
    If Contains(B, x) Then
        Add(R, x)
    End If
End For
Return R
```

End

Function Difference(A: Set, B: Set) -> Set

```
R <- empty set
For each x in A Do
    If Not Contains(B, x) Then
        Add(R, x)
    End If
End For
Return R
```

End

Function Union(A: Set, B: Set) -> Set

```
R <- empty set
For each x in A Do
    Add(R, x)
End For
For each x in B Do
    // Add ignores the duplicates already in A
    Add(R, x)
End For
Return R
```

End

Function Jaccard(A: Set, B: Set) -> Real

```
common <- Size(Intersection(A, B))
// |union| = |A| + |B| - |intersection|, no need to build the union
total <- Size(A) + Size(B) - common
// both sets are empty, so the union is empty too
If total = 0 Then
    coefficient <- 0
Else
    coefficient <- common / total
End If
Return coefficient
```

End

```

// score[g] = number of friends that u and g have in common
// cost: the sum of the degrees of the friends of u, about  $d^2$  for an average degree d
Function SuggestFriends(Friends: Dictionary of Set, u: User) -> Dictionary
    score <- empty dictionary
    For each f in Friends[u] Do
        For each g in Friends[f] Do
            // skip u itself and the people u already knows
            If g != u And Not Contains(Friends[u], g) Then
                If ContainsKey(score, g) Then
                    score[g] <- score[g] + 1
                Else
                    score[g] <- 1
                End If
            End If
        End For
    End For
    Return score
End

```

1. What is the time complexity of set intersection for sets of size m and n?

With hash sets we walk the smaller set and do one Contains for each element: $O(\min(m, n))$ on average. In the worst case (many collisions) a Contains costs $O(n)$, so $O(m * n)$, like the naive double loop on two unsorted arrays.

2. How does the representation (sorted array vs. hash set vs. bit array) affect complexity?

With a sorted array a lookup is a binary search, $O(\log n)$, and intersection, union and difference are a merge of the two arrays with two indices, $O(m + n)$, like in MergeSort; adding a friend costs $O(n)$, because the other elements must be shifted. A hash set makes lookup and insertion $O(1)$ on average, so the intersection costs $O(\min(m, n))$, the union $O(m + n)$ and the difference $O(m)$, with $O(n)$ space plus the empty buckets. A bit array (one bit for each of the N possible ids) also has $O(1)$ lookup and insertion, but it uses N bits for every set. It is very fast on set operations, but time and space depend on the number of possible ids, not on the number of friends.

3. For a network with millions of users, which representation would you choose and why?

Hash sets. A user has only a few hundred friends, but a bit array needs one bit for every user of the network: with 10^6 users that is 125 KB per user, 125 GB in total, almost all zeros. A hash set only stores the real friends, and it finds, adds and removes a friend in $O(1)$.

More complexity analysis and reflection

Intersection is $O(\min(m, n))$ because it walks the smaller set, Difference is $O(m)$ and Union is $O(m + n)$. Jaccard does not build the union, it only uses the size of the intersection, so it is faster and uses less memory.

Test set

Set operations. All the tests print OK. We also compared our results with the set operators of Python (&, - and |).

Case	A	B	Mutual friends	Only A	Only B	Union size	Jaccard
example of the lab	{101, 102, 103, 104, 105}	{103, 104, 106, 107, 108}	{103, 104}	{101, 102, 105}	{106, 107, 108}	8	0.25
no common friend (Roma and Juventus)	{Totti, De Rossi}	{Del Piero, Buffon}	{ }	{Totti, De Rossi}	{Del Piero, Buffon}	4	0
same friends	{Giulia, Marco, Sofia}	{Giulia, Marco, Sofia}	{Giulia, Marco, Sofia}	{ }	{ }	3	1
B inside A (swap)	{Giulia, Marco, Sofia, Luca}	{Marco, Sofia}	{Marco, Sofia}	{Giulia, Luca}	{ }	4	0.5

Case	A	B	Mutual friends	Only A	Only B	Union size	Jaccard
A without friends	{ }	{Nonna Pina, Zio Gino}	{ }	{ }	{Nonna Pina, Zio Gino}	2	0
both without friends	{ }	{ }	{ }	{ }	{ }	0	0

Friend of friend suggestions, on a small network of Italian friends: Marco: {Giulia, Luca}, Giulia: {Marco, Luca, Sofia}, Luca: {Marco, Giulia, Sofia, Chiara}, Sofia: {Giulia, Luca}, Chiara: {Luca}. Nonno Pino has no friends, and Totti, De Rossi and Pellegrini are all friends. The result gives for each suggested user the number of mutual friends.

Case	u	Friends of u	Expected	Got
two candidates	Marco	{Giulia, Luca}	{Sofia: 2, Chiara: 1}	{Sofia: 2, Chiara: 1}
only one friend	Chiara	{Luca}	{Marco: 1, Giulia: 1, Sofia: 1}	{Marco: 1, Giulia: 1, Sofia: 1}
no friends	Nonno Pino	{ }	{ }	{ }
everybody is already a friend	Totti	{De Rossi, Pellegrini}	{ }	{ }

EXERCISE 3 - FRIEND RECOMMENDATION BY COMMON INTERESTS

M is the $U \times I$ user-interest matrix (values from 0 to 10): the row $M[v]$ is the profile of user v , and $\text{Column}(M, j)$ holds the values of interest j for all the U users. U and I are global variables.

Function CosineSimilarity(a: Array, b: Array, n: Integer) -> Real

```
dot <- 0
normA <- 0
normB <- 0
For j <- 0 To n - 1 Do
    dot <- dot + a[j] * b[j]
    normA <- normA + a[j] * a[j]
    normB <- normB + b[j] * b[j]
End For
If normA = 0 Or normB = 0 Then
    // a zero vector is similar to nothing
    similarity <- 0
Else
    similarity <- dot / (sqrt(normA) * sqrt(normB))
End If
Return similarity
End
```

Function TopKSimilarUsers(M: Matrix, u: Integer, friends: Set, K: Integer) -> Array of pairs

```
candidates <- empty array
For v <- 0 To U - 1 Do
    // skip u itself and the users already friends of u
    If v != u And Not Contains(friends, v) Then
        s <- CosineSimilarity(M[u], M[v], I)
        Append(candidates, (v, s))
    End If
End For
Sort candidates by s in descending order
// there can be fewer than K candidates
Return the first min(K, Length(candidates)) elements of candidates
End
```

Function RecommendInterests(M: Matrix, u: Integer, neighbors: Array of pairs) -> Array of pairs

```
recommendations <- empty array
For j <- 0 To I - 1 Do
    // 0 means "not rated": the matrix has no other value for a missing rating
    If M[u][j] = 0 Then
        num <- 0
        den <- 0
        // neighbors holds the pairs (v, s) given by TopKSimilarUsers
        For each (v, s) in neighbors Do
            If M[v][j] > 0 Then
                num <- num + s * M[v][j]
                den <- den + s
            End If
        End For
        If den > 0 Then
            // weighted average of the values of the neighbors
            predicted <- num / den
            Append(recommendations, (j, predicted))
        End If
    End If
End For
Sort recommendations by predicted in descending order
Return recommendations
End
```

```

// two interests are in the same group if their similarity is >= threshold (for example 0.8)
Function GroupInterests(M: Matrix, threshold: Real) -> Array of Set
  groups <- empty array
  assigned <- array of I values, all False
  For j <- 0 To I - 1 Do
    If Not assigned[j] Then
      group <- empty set
      Add(group, j)
      assigned[j] <- True
      For k <- j + 1 To I - 1 Do
        If Not assigned[k] Then
          // a column has U values, one for each user
          If CosineSimilarity(Column(M, j), Column(M, k), U) >= threshold Then
            Add(group, k)
            assigned[k] <- True
          End If
        End If
      End For
      Append(groups, group)
    End If
  End For
  // each group can then become one column of a smaller matrix, the average of its interests
  Return groups
End

```

New in this version: ReduceMatrix, the second part of the bonus. It builds the smaller matrix from the groups.

```

// each group becomes one column of the new U x G matrix, the average of its interests
Function ReduceMatrix(M: Matrix, groups: Array of Set) -> Matrix
  G <- Length(groups)
  R <- new U x G matrix
  For v <- 0 To U - 1 Do
    For g <- 0 To G - 1 Do
      total <- 0
      For each j in groups[g] Do
        total <- total + M[v][j]
      End For
      // a group is never empty, it has at least its first interest
      R[v][g] <- total / Size(groups[g])
    End For
  End For
  Return R
End

```

1. What is the time complexity of computing all pairwise similarities for U users and I interests?

There are about $U^2 / 2$ pairs of users, and each similarity takes $O(I)$ time to compute. So the total is $O(U^2 * I)$.

2. How would you optimize this for sparse matrices (most users have few interests)?

Store only the non-zero values of each user, for example as a list of (interest, value) pairs. Zeros do not contribute to the dot product, so we skip them. The cost of one similarity then depends only on the number of interests the two users actually have, not on I. We can also precompute each user's norm once instead of recomputing it for every pair, and keep for each interest the list of the users that have it (an inverted index): then we only compare the users that share at least one interest.

3. What is the space complexity of storing the full matrix vs. sparse representations?

The full matrix needs $O(U * I)$ space, zeros included. A sparse representation needs only $O(U + \text{nnz})$, where nnz is the number of non-zero values and $O(U)$ is for the start of the list of each user. When most users have few interests, nnz is much smaller than $U * I$.

More complexity analysis and reflection

CosineSimilarity is $\Theta(n)$. TopKSimilarUsers computes $U - 1$ similarities and then sorts them, so it is $O(U * I + U \log U)$. RecommendInterests is $O(I * K)$, GroupInterests is $O(I^2 * U)$, and ReduceMatrix reads every cell once, so it is $O(U * I)$.

With threshold 0.8 the 6 interests become only 2 groups, and the similarity between user 0 and user 1 stays almost the same (0.988 before, 0.992 after), so the smaller matrix still works.

Test set

The tests use this matrix. Users 0, 1 and 2 are the example of the lab, and we added three users for the edge cases. The interests are numbered from 0 (Music) to 5 (Food).

User	Music	Sports	Tech	Fashion	Travel	Food	Note
0	10	0	8	2	5	7	lab example
1	9	1	7	3	6	8	lab example
2	2	9	1	8	3	0	lab example
3	8	0	9	0	4	6	Leonardo da Vinci: close to user 0, no Sports and no Fashion
4	0	0	0	0	0	0	Nonno Pino: no interest
5	0	10	0	9	0	0	Totti: only Sports and Fashion

All the tests print OK. For the example of the lab we get 0.99 and 0.30, not 0.94 and 0.18 as in the lab text, but the result is the same: user 1 is very similar to user 0, user 2 is not.

Case	a	b	Expected	Got
example 1 of the lab	user 0	user 1	0.99	0.99
example 2 of the lab	user 0	user 2	0.30	0.30
same profile	user 0	user 0	1	1.00
zero vector	user 0	Nonno Pino	0	0
no interest in common	Leonardo	Totti	0	0
Mario and Luigi, Luigi gives double values	[1, 2, 3]	[2, 4, 6]	1	1.00

Top K similar users. The result is a list of pairs (v, s), from the most similar user.

Case	u	Friends of u	K	Expected	Got
friend skipped	0	{1}	2	(3, 0.98), (2, 0.30)	(3, 0.98), (2, 0.30)
K bigger than the candidates	0	{1, 2}	10	(3, 0.98), (5, 0.09), (4, 0)	(3, 0.98), (5, 0.09), (4, 0)
K = 0	0	{ }	0	[]	[]
everybody is a friend	0	{1, 2, 3, 4, 5}	3	[]	[]
Nonno Pino, no interest	4	{ }	2	(0, 0), (1, 0)	(0, 0), (1, 0)

Recommended interests. The result is a list of pairs (interest, predicted value). Leonardo (user 3) has not rated Sports (1) and Fashion (3), so the algorithm predicts these two values from his neighbors.

Case	u	Neighbors	Expected	Got
top 2 neighbors	3	(0, 0.98), (1, 0.95)	(3, 2.49), (1, 1.00)	(3, 2.49), (1, 1.00)
top 3 neighbors	3	(0, 0.98), (1, 0.95), (2, 0.21)	(3, 3.03), (1, 2.44)	(3, 3.03), (1, 2.44)
nothing to recommend	1	(0, 0.99), (3, 0.95)	[]	[]
no neighbor	3	[]	[]	[]
Nonno Pino, neighbor with similarity 0	4	(0, 0)	[]	[]

Groups of interests and reduced matrix.

Case	Threshold	Expected	Got
example	0.8	{0, 2, 4, 5}, {1, 3}	{0, 2, 4, 5}, {1, 3}

Case	Threshold	Expected	Got
threshold 0	0	{0, 1, 2, 3, 4, 5}	{0, 1, 2, 3, 4, 5}
threshold bigger than 1	1.1	{0}, {1}, {2}, {3}, {4}, {5}	{0}, {1}, {2}, {3}, {4}, {5}

With threshold 0.8, ReduceMatrix gives the expected rows, for example [7.5, 1] for user 0.

EXERCISE 4 - MUTUAL FOLLOWERS MATRIX

Structure FollowerMatrix

matrix: Array [1..N, 1..N] of Boolean

size: Integer

user_count: Integer

End Structure

Function Create(N: Integer) -> FollowerMatrix

G: FollowerMatrix

G.size <- N

G.user_count <- 0

For i <- 1 To N Do

For j <- 1 To N Do

G.matrix[i][j] <- False

End For

End For

Return G

End

Function AddUser(output G: FollowerMatrix) -> Integer

// -1 means that the matrix is full

id <- -1

If G.user_count < G.size Then

G.user_count <- G.user_count + 1

id <- G.user_count

End If

Return id

End

Function IsUser(G: FollowerMatrix, u: Integer) -> Boolean

Return u >= 1 And u <= G.user_count

End

Procedure Follow(output G: FollowerMatrix, follower: Integer, followee: Integer)

// both users must exist, and nobody can follow themselves

If IsUser(G, follower) And IsUser(G, followee) And follower != followee Then

G.matrix[follower][followee] <- True

End If

End

Procedure Unfollow(output G: FollowerMatrix, follower: Integer, followee: Integer)

If IsUser(G, follower) And IsUser(G, followee) Then

G.matrix[follower][followee] <- False

End If

End

Function Is_following(G: FollowerMatrix, follower: Integer, followee: Integer) -> Boolean

result <- False

// check the users first, so the matrix is never read outside its bounds

If IsUser(G, follower) And IsUser(G, followee) Then

result <- G.matrix[follower][followee]

End If

Return result

End

```
Function Get_followers(G: FollowerMatrix, u: Integer) -> Array
```

```
  // u must be a user (IsUser(G, u) = True)
```

```
  // the followers of u are the True cells of column u
```

```
  result <- empty array
```

```
  For i <- 1 To G.user_count Do
```

```
    If G.matrix[i][u] Then
```

```
      Append(result, i)
```

```
    End If
```

```
  End For
```

```
  Return result
```

```
End
```

```
Function Get_following(G: FollowerMatrix, u: Integer) -> Array
```

```
  // the users that u follows are the True cells of row u
```

```
  result <- empty array
```

```
  For j <- 1 To G.user_count Do
```

```
    If G.matrix[u][j] Then
```

```
      Append(result, j)
```

```
    End If
```

```
  End For
```

```
  Return result
```

```
End
```

```
Function MutualFollows(G: FollowerMatrix) -> Array of pairs
```

```
  // every mutual pair only once, with i < j: (1, 2) and (2, 1) are the same pair
```

```
  pairs <- empty array
```

```
  For i <- 1 To G.user_count Do
```

```
    For j <- i + 1 To G.user_count Do
```

```
      If G.matrix[i][j] And G.matrix[j][i] Then
```

```
        Append(pairs, (i, j))
```

```
      End If
```

```
    End For
```

```
  End For
```

```
  Return pairs
```

```
End
```

```
Function InfluenceScore(G: FollowerMatrix, u: Integer) -> Real
```

```
  // u must be a user, so user_count >= 1 and there is no division by zero
```

```
  Return (Length(Get_followers(G, u)) + Length(Get_following(G, u))) / G.user_count
```

```
End
```

1. What is the time complexity of Get_followers() and Get_following()?

Theta(n), with $n = \text{user_count}$: to find the followers or the followed users of u we have to read the whole column or row of u, even if it contains only False. Follow, Unfollow and Is_following instead read or write a single cell, so they are $O(1)$.

2. What is the space complexity of the adjacency matrix for N users?

Theta(N^2), because the matrix has one cell for every pair of users. The memory does not depend on how many follow relations there are: an empty network takes as much space as a full one.

3. At what network size does this become impractical? Why?

Already with about 10^5 users the matrix has 10^{10} cells, more than 1 GB even with one bit per cell. Moreover real networks are sparse: every user follows only a few hundred people, so almost all the cells are False and the memory is wasted. For big networks it is better to use an adjacency list, which stores for each user only the real followers and followed users.

More complexity analysis and reflection

Follow, Unfollow and Is_following are $O(1)$, while Create is Theta(N^2) because it fills the whole matrix. Get_followers, Get_following and InfluenceScore are Theta(n), and MutualFollows is Theta(n^2) because it checks every pair of users.

Test set

The first tests use the example of the lab, with 1 = Romeo, 2 = Giulietta and 3 = Paride. The lab text gives influence 0/3 for user 3, but Giulietta follows Paride, so we get 1/3. Then we test the edge cases with other users (Mario, Luigi and Peach, and the 4 Ninja Turtles). All the tests print OK.

Case	Operations	Expected	Got
lab example	Get_followers(Paride)	[2]	[2]
lab example	Get_following(Giulietta)	[1, 3]	[1, 3]
lab example	Get_following(Paride)	[]	[]
lab example	MutualFollows()	[(1, 2)]	[(1, 2)]
lab example	InfluenceScore of Romeo, Giulietta and Paride	2/3, 3/3, 1/3	0.667, 1.0, 0.333
full matrix	AddUser()	-1	-1
self follow	Follow(Romeo, Romeo), then Is_following(Romeo, Romeo)	False	False
user that does not exist	Follow(1, 4), then Is_following(1, 4)	False	False
user 0	Is_following(0, 1)	False	False
follow two times	Follow(Romeo, Giulietta) again, then Get_followers(Giulietta)	[1]	[1]
unfollow	Unfollow(Giulietta, Romeo), then MutualFollows()	[]	[]
unfollow when not following	Unfollow(Romeo, Paride), then Get_following(Romeo)	[2]	[2]
size 5, 3 users	AddUser() for Peach, the third user	3	3
size 5, 3 users	Mario and Luigi follow each other, MutualFollows()	[(1, 2)]	[(1, 2)]
user without relations	InfluenceScore(Peach)	0	0.0
Ninja Turtles, n = 4	number of mutual pairs	6	6
Ninja Turtles, n = 4	InfluenceScore(Leonardo)	1.5	1.5

FINAL QUESTION

Give some potential usage of these LAB 2 developed algorithms in your future semester project.

We still do not know the details of the semester project, but we think we could reuse these algorithms:

- exercise 1: read a text only once to count things, for example to check user messages or a log file;
- exercise 2: hash sets to check quickly if an element is already there, for example the visited nodes in a graph, and the Jaccard coefficient to find similar users or items;
- exercise 3: the cosine similarity and the top K to compare users or products and make recommendations;
- exercise 4: the adjacency matrix to store a small graph and find the neighbours of a node, for example in BFS or DFS; for a big graph we would use adjacency lists.